

IBM BOB 2.0 HACKATHON · SEPTEMBER 2026

Receipts

Receipts for what your AI agent actually did. An **application-maintenance** tool for AI-authored code — every task in this build ran in IBM Bob's **Agent mode**.

BUILT BY

Vishal Patel

LIVE

<https://receipts-black-five.vercel.app>

REPO

<https://github.com/ishal1410/receipts>

LICENCE

MIT

A human spent 9,567 milliseconds approving code that never reached a single commit.

Nobody saw it happen. That is the whole problem.

CALC.PY — WRITTEN, APPROVED, GONE		
A	Bob wrote & applied the guard, sized at	65 bytes
B	Commits ever made to calc.py	3
A	Of those, containing the guard	0
	Human approval spent on it	9,567 ms

WHAT EVERY TOOL REPORTED	
01	The tool call succeeded .
02	The approval was logged .
03	The edit was applied to disk .
It never reached a single commit — not that one, not any of the three. Every system in the chain still reports that task as a success.	
The code died before it ever shipped.	

Before the agent reads your prompt, 19,009 tokens are already spent.

Skill definitions load into every task's budget — measured with zero skills loaded.

TOKENS CONSUMED BEFORE THE FIRST INSTRUCTION

19,009

Skill definitions, loaded into the context window of every task — while `loadedSkills: []` sits empty in the same export. Present in all **six task exports**, running 18,572–19,009 tokens — a range, not a constant. Today no export surfaces it to you.

Receipts answers three questions Bob's own export cannot.

Per task: what it cost, what survived, where the budget went.

PER TASK, ON ONE SCREEN

01 Bobcoins spent — the task's own metered cost, read from the export.

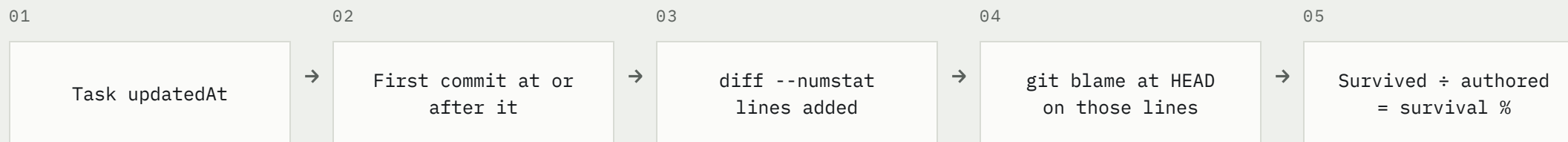
02 Did the code survive to HEAD — per line, not per file.

03 Where the context budget went — including the overhead on slide 3.

Scope, stated plainly: reads what Bob exports today — not yet the full local
~/ .bob/db/bob.db task archive.

gitSha is always null. So the join runs on time — and on the lines themselves.

Five steps, no guessing: a task's edits either survive to HEAD or they don't.



TWO THINGS THIS DOES NOT DEPEND ON

It reads Bob's **exported** task JSON — not any internal IBM attribution store, so nothing here waits on a setting being switched on. And the time window alone is never enough: step 4 is an **authorship test**, so a human's commit landing in the same window cannot be credited to Bob. *Renames register as 0% survival — a documented limit, not a hidden one.*

No backend. Nothing to go down during judging.

A snapshot script, a static JSON file, a static app.

BUILD TIME

`tools/snapshot.mjs`
reads git + exports



ARTEFACT

`public/analysis.json`
one committed file



RUNTIME

Static React app
reads only that file

WHAT THIS BUYS

The browser **never touches git**. One JSON file is the entire runtime dependency, and every number on screen traces back to it — never retyped by hand. Pull the network out after load and the page still renders the same numbers.

The survival numbers are computed **once**, at snapshot time, and frozen into a committed file.

Runtime dependencies: 1 file

Bob is the only input format. Bob also built this.

Every task in this repo ran in Bob's Agent mode — and the handoff runs both ways.

BOB, THREE WAYS

IN **Input:** Bob's exported task-session JSON. Nothing else is parsed.

BY **Build:** every task run in Bob's **Agent mode**, exported and committed.

OUT **Back to Bob:** Receipts emits a remediation prompt for you to paste into Bob. Nothing here has been pasted back; the return leg is built, not exercised.

THE WORKFLOW THIS IMPROVES

Application maintenance — surfacing rework signal on AI-authored code after the agent has already moved on.

Auditable in the repo: `bob_sessions/` holds 2 task exports, 2 markdown reports and 2 session screenshots — the receipts for building Receipts.

It doesn't just report the rot. It ends on an action.

A deterministic rule engine — no model call, no API key — that hands you a prompt.

DETERMINISTIC, NOT GENERATED

Fixed rules decide what to say — one is implemented, R2, not the four the spec sketches. **No model call** sits in the path, so the same input gives the same output every time — and the judge can reproduce it. Each flagged task ends on one button:

Copy prompt for Bob

That prompt is yours to paste into Bob. **No loop has closed here** — on this corpus the rule fired once, and the page says outright that nothing has been sent.

The buyer is the VP Engineering defending agentic-coding spend at renewal.

Not developers. Not hackathon entrants. The person who answers for the line item.

ASSUMPTIONS — NUMBERED, NAMED, UNHIDDEN			
A1	paid agentic-coding seats, 2026	4,000,000	ESTIMATE ±50%
	basis: GitHub reported >1.8M paid Copilot subscribers (2025); plus Cursor, Claude Code, Windsurf, Bob. Rounded.		
A2	entry tier (Team)	\$60/seat/yr	OUR PRICE
	our price, not market data.		
A3	share in orgs with a renewal review	25%	ASSUMPTION
	no survey behind it.		

WHAT THAT BUYS	
TAM	4.0M × \$60
\$240M/yr	
SAM	4.0M × 0.25 × \$60
\$60M/yr	
SOM	1.0M × 1.5% × \$60
\$900K ARR	
at month 36 · 30 accounts × 500 devs	

“These are assumptions with names on them.”

A1-A3 are estimates, not findings. Every measured number in this deck is on slides 2-8.

It has to cost less than the rework it redirects. Break-even is 0.257% of tasks.

Left: what we charge. Right: the arithmetic the buyer runs.

PRICE		
Open	free	CLI parser, single export, local
Team	\$5/dev/mo	25-seat floor
Enterprise	\$8/dev/mo	SSO, policy gates, CI check

THE BUYER'S ARITHMETIC – 500-DEV ORG	
500 devs · 6 agent edit tasks/dev/day	ESTIMATE · 220 days = 660,000 tasks/yr
Receipts Enterprise, 500 × \$96	\$48,000 /yr
Redirect 1% of all tasks – 20 min at \$85/hr	ESTIMATE :
6,600 × 20 min = 2,200 hrs × \$85	\$187,000 /yr
BREAK-EVEN REDIRECT RATE	0.257%
\$96/dev/yr is 0.06% of that developer's cost. The audited Bob spend is a separate line: \$94,069 metered/yr.	

“At \$96 a developer a year — a tenth of one percent of what that developer costs — this pays for itself at fewer than three redirected tasks in a thousand.”

0.285058 coins/task is measured, not modelled — from the Bob exports in bob_sessions/.

Five categories record this. None of them asks *why*.

	PER-CHANGE ATTRIBUTION	COST ATTACHED	REWORK SIGNAL	CONTEXT BUDGET	ANSWERS WHY
git blame / bisect	author only	—	manual	—	WHEN, not WHY
Copilot / Cursor dashboards	aggregate	org total	—	—	—
LinearB · Faros · DX · Jellyfish	org metrics	—	team-level	—	—
SLSA · SBOM · in-toto	starts at source	—	—	—	build, not prompt
IBM Bob, built in (shipped, off)	line-level, native	—	drops vanished lines	—	records, never asks
Receipts	task → file → line	coins per task	survival to HEAD	measured	prompt + tool calls

WHERE THEY BEAT US TODAY

git

 needs no export and is universal.

Dashboards

 ship SSO, RBAC and org rollups we do not have.

DX and Jellyfish

 have cross-company benchmark data we have none of.

SLSA

 has a standard and an ecosystem; we have a parser.

THE TOP ROW IS IN THEIR BINARY, NOT THEIR ROADMAP

extension.js

 ships

AttributionLogStore

, a survival check logging “*Contribution no longer present, dropping log*”, and git-notes at

refs/notes/bob

 — table created by

002_attribution

, IBM’s second migration ever. **Zero rows**, behind one flag:

commitCoAuthor: false

.

IBM shipped the ledger and left it switched off. Receipts turns it on and asks it a question.

A receipt is only worth writing if something downstream reads it.

30 DAYS

A **CI check** that fails a PR whose agent attribution is unresolvable.

90 DAYS

The same parser across **Bob, Copilot, Cursor and Claude Code**, normalised into one receipt schema.

12 MONTHS

Take that schema to the **SLSA working group** as the missing upstream link.

Every number in this deck came out of the Bob sessions that built this project.

LIVE

<https://receipts-black-five.vercel.app>

REPO

<https://github.com/ishal1410/receipts>

LICENCE

MIT